Architecture Document

SheStarts AI Career Counselor

_

1. System Design

1.1 Overview

SheStarts AI Career Counselor is a Streamlit web app built to support women restarting their careers. The architecture combines a simple web interface, a local relational database, AI workflow orchestration, and downloadable output generation.

1.2 System Components

- `app.py`: Main Streamlit app entrypoint — manages navigation, page rendering, and state.
- `config/config.py`: Environment and API key configuration loader.
- `database/db.py`: SQLAlchemy engine and session factory.
- `database/models.py`: User and CareerProfile ORM models.
- `database/init_db.py`: Database initialization helper.
- `services/auth_service.py`: Handles user registration, authentication, and session state.
- `services/database_service.py`: Saves profiles, loads user-specific career data, and queries reports.
- `services/gemini_service.py`: Sends prompts to Google Gemini and receives generated text.
- `services/resume_service.py`: Extracts resume text from uploaded PDF files.
- `services/pdf_service.py`: Creates downloadable career report PDFs.
- `services/report_visualization.py`: Converts AI reports into dashboard widgets and summaries.
- `services/counselor_visuals.py`: Generates visual guidance content for the counselor page.
- `agents/assessment_agent.py`: Constructs prompts for career report generation.
- `agents/counselor_agent.py`: Constructs prompts for profile-aware counselor chat.
- `prompts/prompts.py`: Central prompt templates and output formatting instructions.

1.3 Interaction Layers

- **Presentation:** Streamlit pages and UI components.
- **Business Logic:** Service layer managing workflows, validation, and persistence.
- **AI Layer:** Gemini prompt construction and API communication.
- **Storage:** SQLite database storing user accounts and career profiles.

1.4 System Interaction Diagram

```
User Browser --> Streamlit App (app.py)
|
|-- auth_service.py --> SQLite users
|-- database_service.py --> SQLite career_profiles
|-- assessment_agent.py --> gemini_service.py --> AI report
|-- resume_service.py --> PDF resume text extraction
|-- counselor_agent.py --> gemini_service.py --> chat response
|-- pdf_service.py --> PDF export
```

2. AI Workflow

2.1 Career Assessment Workflow

1. User submits the Career Assessment form.
2. The app packages user input into a structured payload.
3. assessment_agent generates a report prompt with user context and desired output structure.
4. gemini_service sends the prompt to Gemini and receives a career report.
5. The career report is saved to the database by database_service.
6. The dashboard renders the AI report and extracts summary metrics.

2.2 Resume Feedback Workflow

1. User uploads a resume PDF.
2. resume_service extracts the resume text.
3. The app includes resume text with assessment data in the AI prompt.
4. Gemini returns resume-aware guidance and skill recommendations.

2.3 Counselor Chat Workflow

1. User opens the Career Counselor page.
2. The app retrieves the logged-in user's saved profile and latest assessment.
3. build_counselor_context composes profile, goals, and resume snippets.
4. counselor_agent creates a prompt asking Gemini to answer with personalised context.
5. Gemini returns a conversational, actionable response tailored to the user.

3. Data Flow

3.1 User Data Lifecycle

- **Login / Register:** User credentials and basic profile are stored in users.
- **Assessment:** Career details and AI report results are stored in career_profiles.
- **Dashboard:** Only current-user data is retrieved and displayed.
- **Chat:** Profile data is used to personalize counselor responses.
- **Report Export:** Latest stored profile data is converted into a PDF.

3.2 Data Flow Diagram

```
User --> app.py
--> auth_service.py --> SQLite users
--> assessment_agent.py --> prompts/prompts.py
--> gemini_service.py --> Gemini AI
--> database_service.py --> SQLite career_profiles
--> report_visualization --> dashboard
--> counselor_agent.py + saved profile --> Gemini --> chat
```

3.3 Privacy and Ownership

- Each career profile is linked to a specific authenticated user.
- Dashboard and chat load only the current user's records.
- Exported PDF content is generated from the current user's own saved data.

4. Prompt Design

4.1 Report Prompt Design

Prompts are defined in `prompts/prompts.py` to enforce consistent structure. They request:

- Career strengths and re-start readiness.
- Recommended skills and confidence-building actions.
- A 30/60/90 day action plan.
- A final summary and next steps.

Structure is intentionally markdown-like for easier parsing by the app.

4.2 Chat Prompt Design

Counselor prompts emphasise:

- A supportive tone.
- Actionable, concise advice.
- Explicit use of user profile and resume context.
- Practical next steps tied to the user's goal.

The prompt instructs Gemini to avoid generic answers and remain relevant to the user's background.

5. Technology Choice

- **Python:** Primary language for the app and all services.
- **Streamlit:** Rapid prototyping UI framework.
- **SQLAlchemy + SQLite:** Simple relational persistence for prototype and demos.
- **Google Gemini:** Main AI generation engine (google-generativeai).
- **PyMuPDF:** Extracting text from uploaded resume PDFs.
- **ReportLab:** Creating formatted PDF exports.
- **Pandas:** Summary data formatting and lightweight processing.

6. Notes

The architecture is designed for rapid development with a focus on clear user flows and AI integration. Production readiness would require:

- Moving away from SQLite to a production-grade database (e.g. PostgreSQL).
- Adding a dedicated backend/API layer.
- Applying stronger authentication and session management.
- Implementing structured logging and monitoring.